1)-> Write a program in java to find the Armstrong or palindrome?

Algorithm:

1. Armstrong Check (isArmstrong)

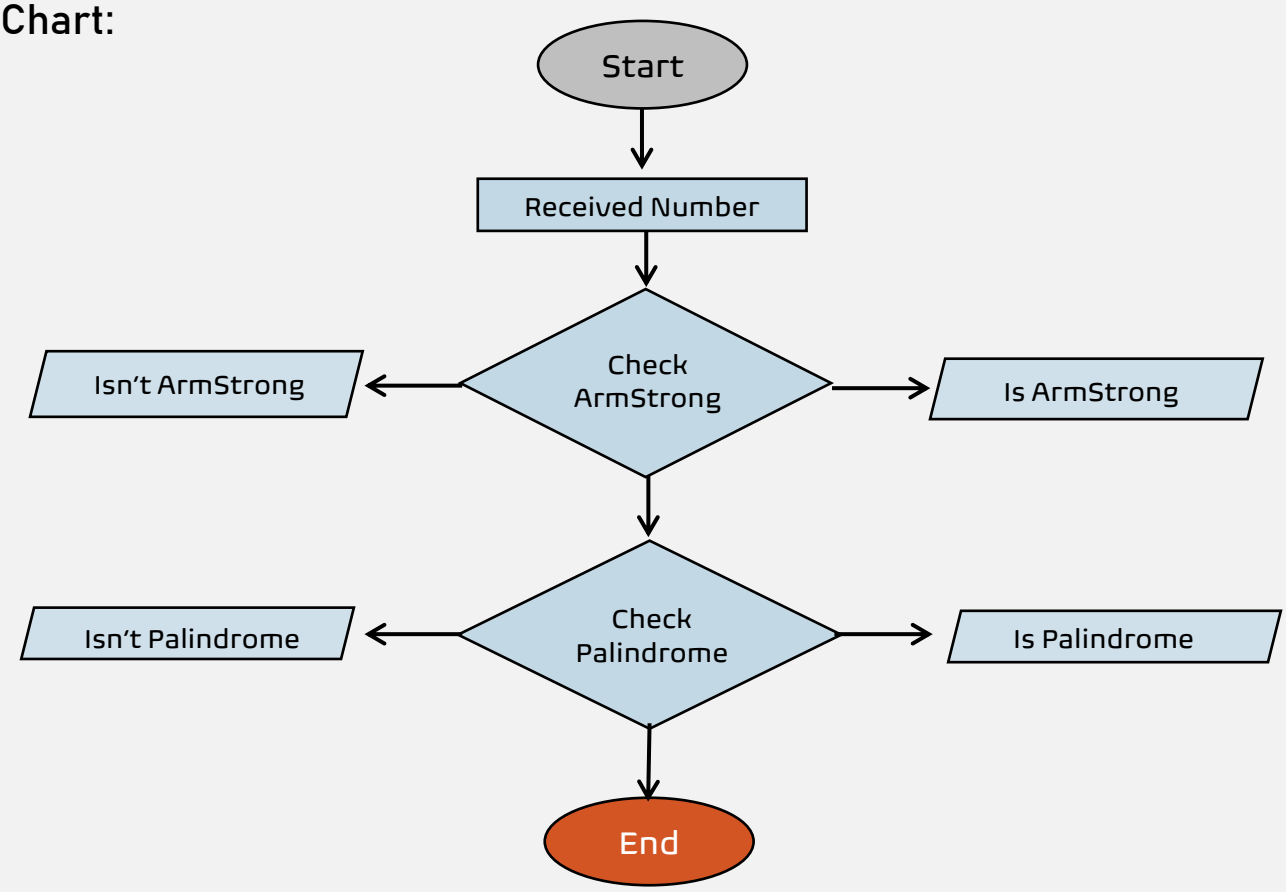
1. Find N: Get the number of digits (N).
2. Initialize Sum: Sum = 0; Original = Number.
3. Loop: While Number > 0:
    - Digit = Number (mod 10).
    - Sum += \text{Digit}^N.
    - Number = 10.
4. Result: Return Sum = Original.

---

2. Palindrome Check (isPalindrome)

1. Initialize Reverse: Reverse = 0; Original = Number.
2. Loop: While Number > 0:
    - Digit = Number} mod{10}.
    - Reverse = Reverse X 10 +Digit.
    - Number /= 10.
3. Result: Return Original = Reverse.

FlowChart:



```java
import java.util.Scanner;

public class ArmstrongOrPalindrome {

    // Function to check if a number is an Armstrong number
    public static boolean isArmstrong(int n) {
        int input = n;
        int sum = 0;
        int len = String.valueOf(n).length();

        while (n != 0) {
            int ones = n % 10;
            sum += Math.pow(ones, len);
            n /= 10;
        }

        return sum == input;
    }

    // Function to check if a number is a Palindrome
    public static boolean isPalindrome(int n) {
        int input = n;
        int reverse = 0;

        while (n != 0) {
            int last = n % 10;
            reverse = reverse * 10 + last;
            n /= 10;
        }

        return input == reverse;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input from the user
        System.out.print("Enter a number: ");
        int n = scanner.nextInt();

        // Check Armstrong
        if (isArmstrong(n)) {
            System.out.println(n + " is an Armstrong number.");
        } else {
            System.out.println(n + " is not an Armstrong number.");
        }

        // Check Palindrome
        if (isPalindrome(n)) {
            System.out.println(n + " is a Palindrome.");
        } else {
            System.out.println(n + " is not a Palindrome.");
        }

        scanner.close();
    }
}
```

OutPut:

```
Enter a number: 21312
21312 is not an Armstrong number.
21312 is a Palindrome.
```

```
Enter a number: 407
407 is an Armstrong number.
407 is not a Palindrome.
```

Variable Declaration Table:

Variable Name	**Data Type**	**Description**
n	int	Stores User provided input.
input	int	Saves the value of n for future comparison.
sum	int	Stores the sum of digits for comparison
len	int	Total length of the Number.
last	int	Last number of the Digits
reverse	int	Stores the reverse value of

2)-> Write a program in java to show a null pointer exceptation?

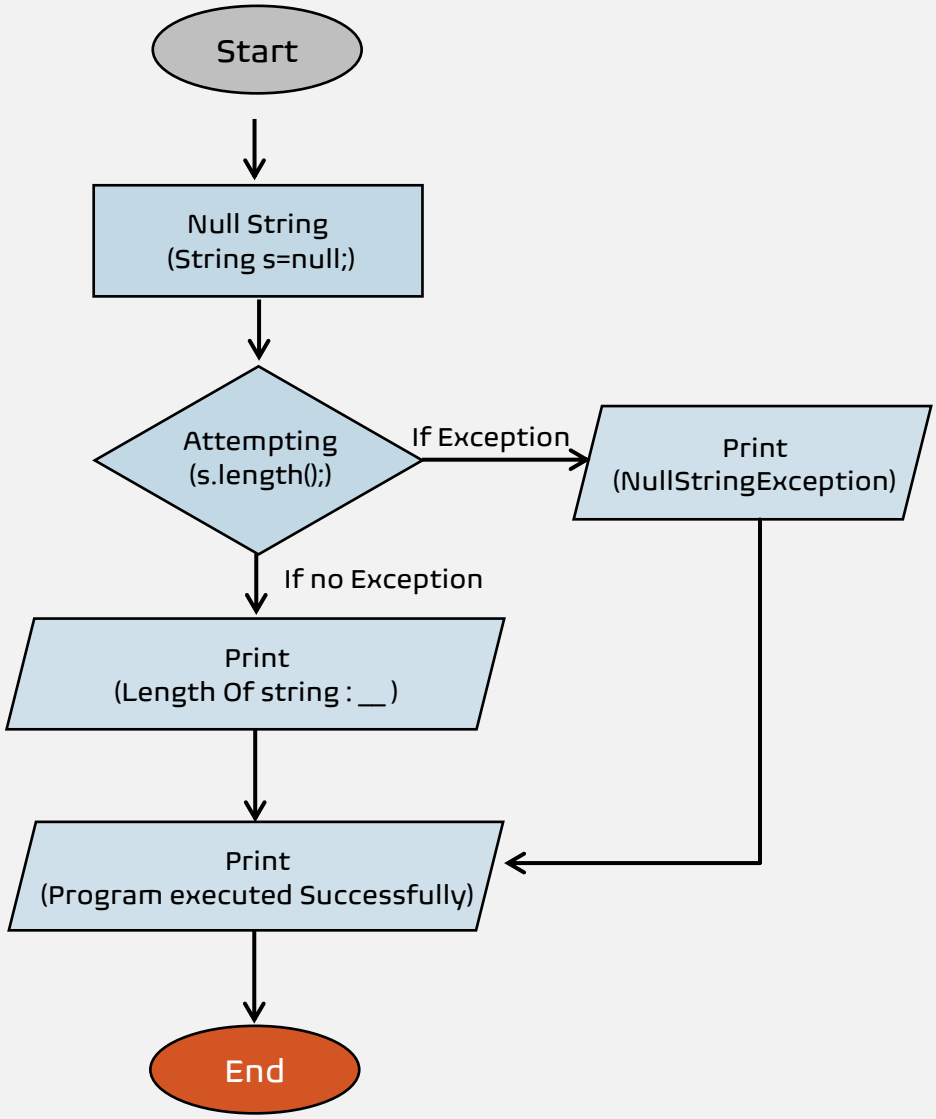
Algorithm:

i. Initialize: Declare a String reference s and set it to null.
ii. Try Block: Attempt to execute s.length().
iii. Catch Block: If Step 2 fails (because s is null):

   - Catch the NullPointerException (NPE).
   - Print the NPE message.

iv. Finish: Print the confirmation message: "Program Executed, Successfully...".

FlowChart:



```java
public class NullString {

    public static void main(String[] args) {
        String s = null; // Initialize a String reference to null

        try {
            // Attempting to call a method on a null object
            System.out.println("The length of the string is: " + s.length());
        } catch (NullPointerException e) {
            // Handling the NullPointerException
            System.out.println("\nCaught a NullPointerException!");
            System.out.println("\nException Message: " + e.getMessage());
        }

        // Program continues normally
        System.out.println("\nProgram Executed, Successfully...\n");
    }
}
```

OutPut:

```
Caught a NullPointerException!

Exception Message: null

Program Executed, Successfully...
```

Variable Declaration Table:

Variable Name	Data Type	Description
s	string	String that holds Null
e	Exception	Exception object represents NullPointerException

3)-> Write a program in java to reverse a given string and replace all Uppercase A into 7, Lowercase a into 3 and print both the output?

Algorithm:

i. Input: Read the user's input string and store it in ST.

ii. Reverse: Create a new string TS by reversing ST.

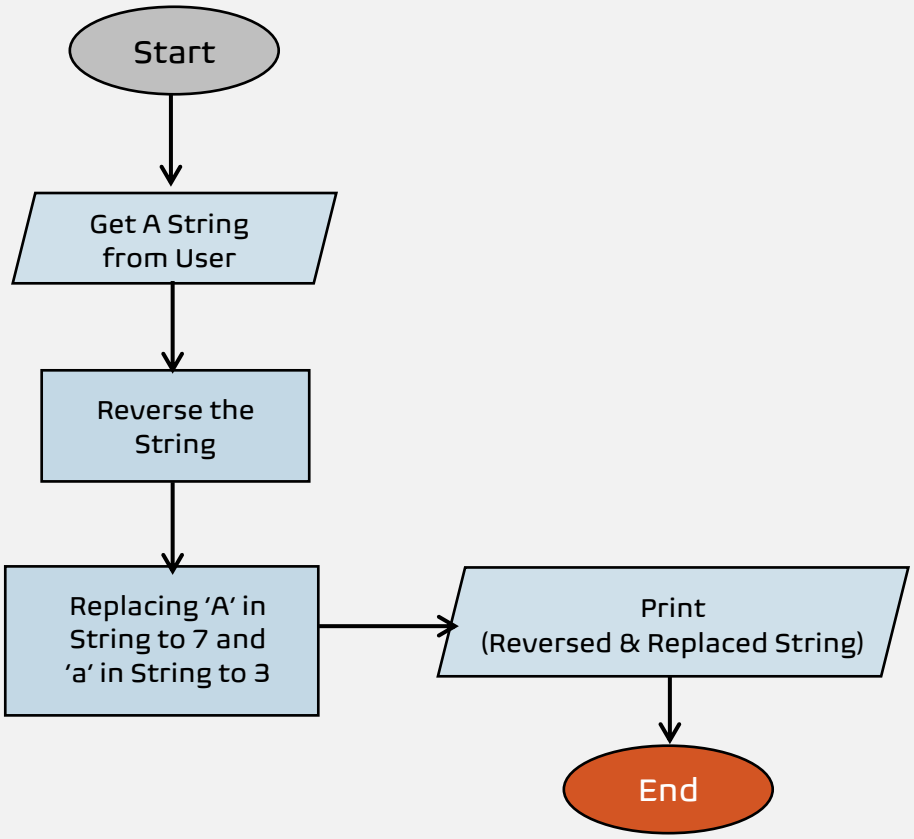
iii. Replace: Create a new string S1 by applying two replacements to TS:

- Replace all occurrences of the character 'A' with '7'.
- Replace all occurrences of the character 'a' with '3'.

iv. Output: Print both the reversed string TS and the reversed and replaced string S1.

v. Clean Up: Close the Scanner.

FlowChart:



```java
import java.util.Scanner;
public class Replace {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        // Input string
        System.out.print("Enter a string: ");
        String ST = s.nextLine();

        // Reverse the string
        String TS = new StringBuilder(ST).reverse().toString();

        // Replace 'A' with '7' and 'a' with '3' in the reversed string
        String S1 = TS.replace('A', '7').replace('a', '3');

        // Print both outputs
        System.out.println("\nReversed String: " + TS);
        System.out.println("Reversed and Replaced String: " + S1+"\n");
       s.close();
    }
}
```

OutPut:

```
Enter a string: FAST forward

Reversed String: drawrof TSAF
Reversed and Replaced String: dr3wrof TS7F
```

Variable Declaration Table:

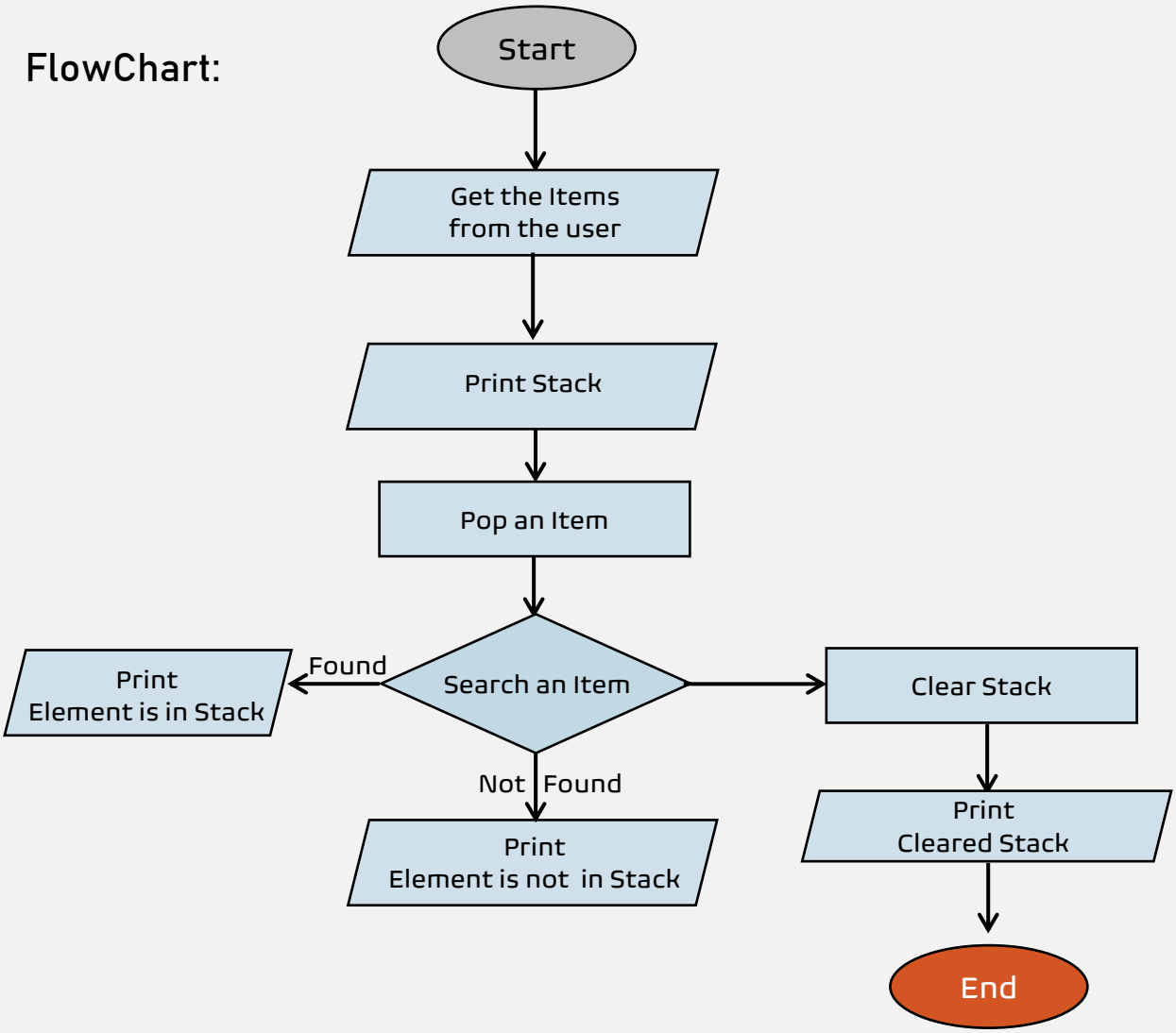
Variable Name	Data Type	Description
ST	string	Input Value of String
TS	string	Reversed Value of String
S1	string	Replaced Value of String

4)-> Write a program in java that demonstrate the use of stack?

Algorithm:

i. Initialize: Create a Scanner sn and an empty Stack of Strings s.
ii. Input Push Count: Read the number of items n to push onto the stack.
iii. Push Items: Loop n times: Read an input item and push it onto s .
iv. Pop Item: Print the current stack s . Then, pop the top item from s and store it in Item .
v. Output Pop: Print the popped item and the updated stack s .
vi. Check Element: Read an element elem) from the user to search for.
vii. Search: Use s.contains(elem) to check if the element is present, and print the result.
viii. Clear: Clear all elements from the stack s .
ix. Finish: Print the cleared stack and close the Scanner.

FlowChart:



```java
import java.util.*;
public class Stackcheck {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);

        // Create a new stack
        Stack<String> s = new Stack<>();

        // Push items onto the stack
        System.out.println("Enter number of items to push onto stack: ");
        int n = sn.nextInt();
        sn.nextLine(); // Consume the newline character

        System.out.println("Enter the items: ");

        for (int i = 0; i < n; i++) {
            String item = sn.nextLine();
            s.push(item);
        }


        System.out.println("Stack: " + s);  // Print the stack

        // Pop an item off the stack

        String Item = s.pop();
        System.out.println("Popped item: " + Item);

        System.out.println("Stack: " + s);  // Print the stack again

        // Check if an item is on the stack

        System.out.println("Check if an item is in Stack : ");
        String elem = sn.nextLine();

        if (s.contains(elem)) {

            System.out.println(elem + " is in the stack.");

        } else {

            System.out.println(elem + " is not in the stack.");

        }
        // Clear the stack

        s.clear();
        System.out.println("Stack: " + s+" after clearing");  // Print the stack again
        sn.close();
    }
}
```

OutPut:

```
Enter number of items to push onto stack:
5
Enter the items:
Desert
Ocean
Forest
River
PLain
Stack: [Desert, Ocean, Forest, River, PLain]
Popped item: PLain
Stack: [Desert, Ocean, Forest, River]
Check if an item is in Stack :
Ocean
Ocean is in the stack.
Stack: [] after clearing
```

Variable Declaration Table:

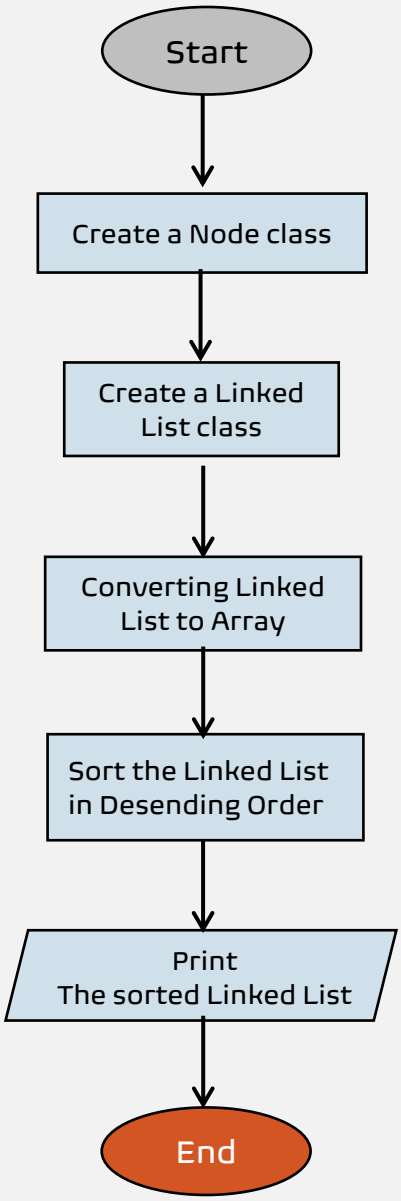
Variable Name	**Data Type**	**Description**
sn	scanner	Object of sccanner class
s	stack	Stack that stores the items
n	int	Number of the elements To be added in Stack
item	String	Used to store values in stack inside For Loop
Item	String	Popped Item of Stack
elem	String	Stores the value given by user to search in Stack

5)-> Write a java program in singly linked list that take input of 7 values and print in descending order.

Algorithm:

i. Create Node Class
ii. Create Linked List Class
iii. Insert 7 Values
iv. Convert Linked List to Array
v. Sort the linked list in descending order
vi. Print the sorted Linked List
vii. End

FlowChart:



```java
import java.util.Scanner;
public class Singly {
     public static void main(String[] args){
         Scanner sc = new Scanner(System.in);
         SinglyLinkedList list = new  SinglyLinkedList();
         System.out.println("Enter 7 values  to insert into the Linked list : ");

         //Input 7 values and insert them into linked list

         for(int i =0; i<7;i++){
             int value = sc.nextInt();
             list.insert(value);
         }

     //Sort the linked list  in decending order
     list.sort();


     //print the linked list in decending order

     System.out.println("Linked list  in desending  order : ");
     list.print();
     sc.close();
    }
}

class Node{

    // Node class  for the singly linked list
    int data;
    Node next ;
     // Constructor to initialize a node with data
    public Node(int data){
        this.data=data;
        this.next=null;
    }
}

   //Linked list class
 class SinglyLinkedList{
     Node head;
```

```
    // method to insert a new value into the linked list
    public void insert(int data){
        Node newNode = new Node (data);
        if (head==null){
            head=newNode;
        }
        else{
            Node temp = head;
            while(temp.next != null){
                temp=temp.next;
            }
            temp.next=newNode;
        }
    }

    //method tos ort the linked list in descending order using bubble tosort
    public void sort(){
//If the list is  empty or has one element
        if(head==null || head.next==null) return;
         for(Node i=head; i!=null; i=i.next){
            for(Node j = i.next;j !=null; j=j.next){
                if(i.data<j.data){
                    //swap data of nodes i and j
                    int temp = i.data;
                    i.data=j.data;
                    j.data=temp;
                }
            }
        }
    }

    // method to print the linked list
    public void print(){
        Node temp=head;
        while(temp !=null){
            System.out.print(temp.data+ " ");
            temp = temp.next;
        }
        System.out.println();
    }
}
```

OutPut:

```
Enter 7 values  to insert into the Linked list :
23
35
65
45
12
25
42
Linked list  in desending  order :
65 45 42 35 25 23 12
```

Variable Declaration Table:

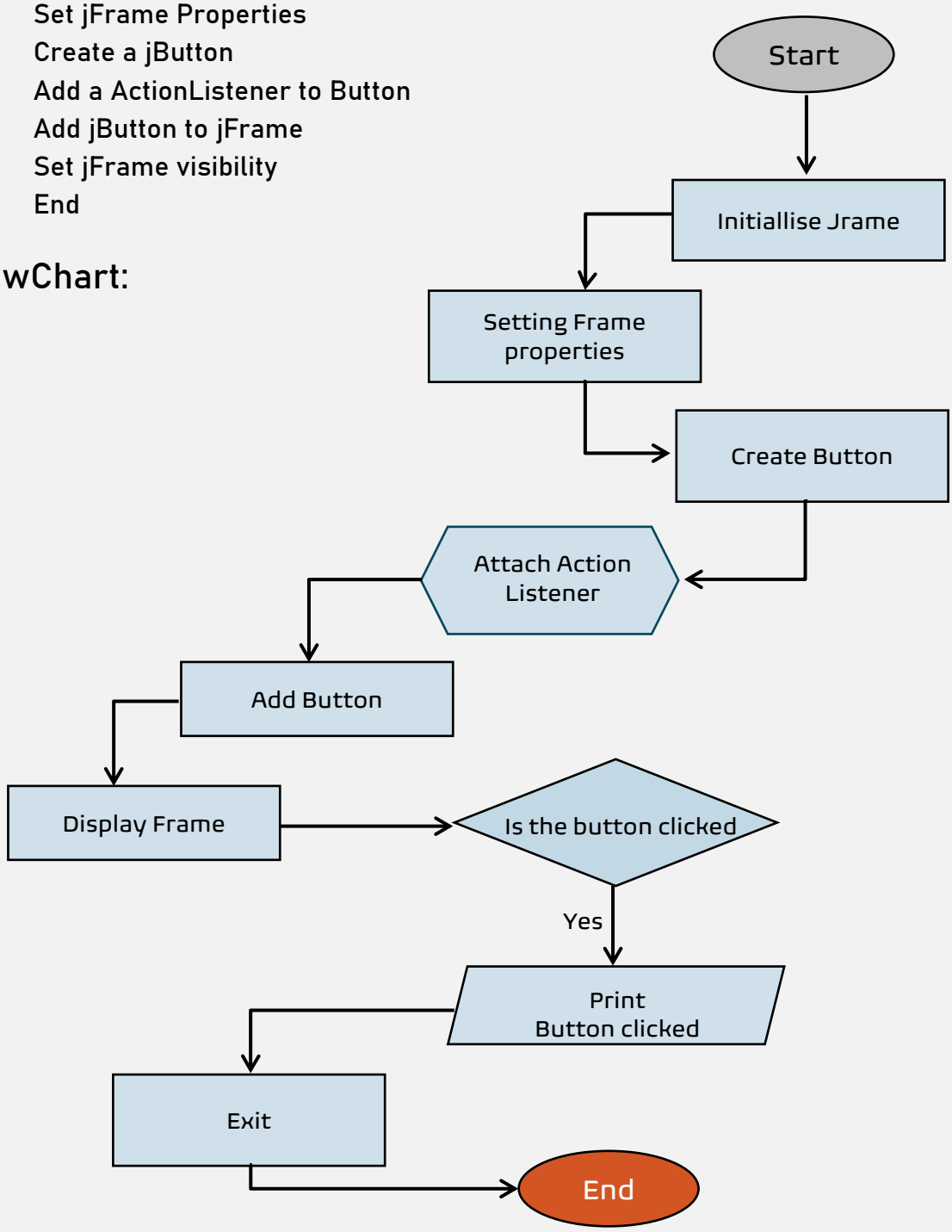
Variable Name	**Data Type**	**Description**
head	Node	Head of linked list
data	int	Data Stored in each node
next	Node	Reference to next node
temp	Node	Temporary Node reference
i	int	To control the loop
j	int	To control sorting loop
swap	Node	Temporary node to swapping

6)-> Write a java program to swing that creates a basic window button.

Algorithm:

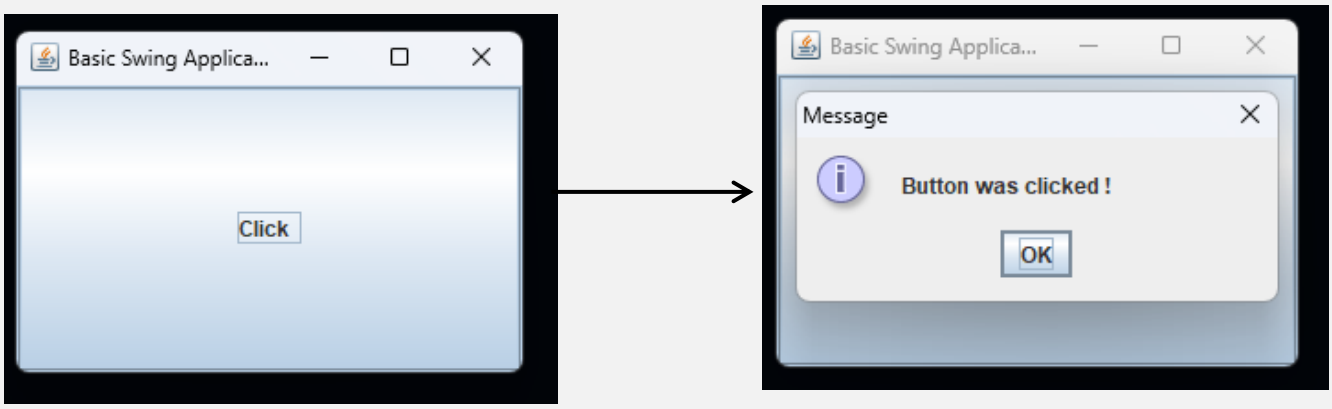
i. Start
ii. Create a jframe
iii. Set jFrame Properties
iv. Create a jButton
v. Add a ActionListener to Button
vi. Add jButton to jFrame
vii. Set jFrame visibility
viii. End

FlowChart:



```java
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class BasicSwingApp {

    public static void main(String[] args) {
        // Create a new frame
        final JFrame frame = new JFrame("Basic Swing Application");

        // Set the default close operation
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Set the size of the frame
        frame.setSize(300, 200);

        // Center the frame on the screen
        frame.setLocationRelativeTo(null);

        // Create a button
        JButton button = new JButton("Click  ");

        // Add an ActionListener to the button
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Show a message dialog when the button is clicked
                JOptionPane.showMessageDialog(frame, "Button was clicked !");
            }
        });

        // Add the button to the frame
        frame.getContentPane().add(button); // Add the button to the content pane

        // Set the frame visibility to true
        frame.setVisible(true);
    }
}
```

OutPut:



Variable Declaration Table:

Variable Name	Data Type	Description
frame	JFrame	Represents the main window application
Button	JButton	Represent the button that the user clicks
ActionListener	Anonymous	An interface that listens for button click events

7)-> Write a program in java to implement following function:

i. forEach( )
ii. trim( )
iii. isequal( )
iv. toUpper( )
v. isNumberOrNot( )

Algorithm:

I. Define the function forEachExample(): Accept a list of strings and use the forEach() method to print each string in the list.

II. Define the function trimExample(): Accept a string, apply the trim() method to remove leading and trailing spaces, and return the result.

III. Define the function isEqualExample(): Accept two strings, use equals() to compare them, and return true if they are equal, otherwise false.

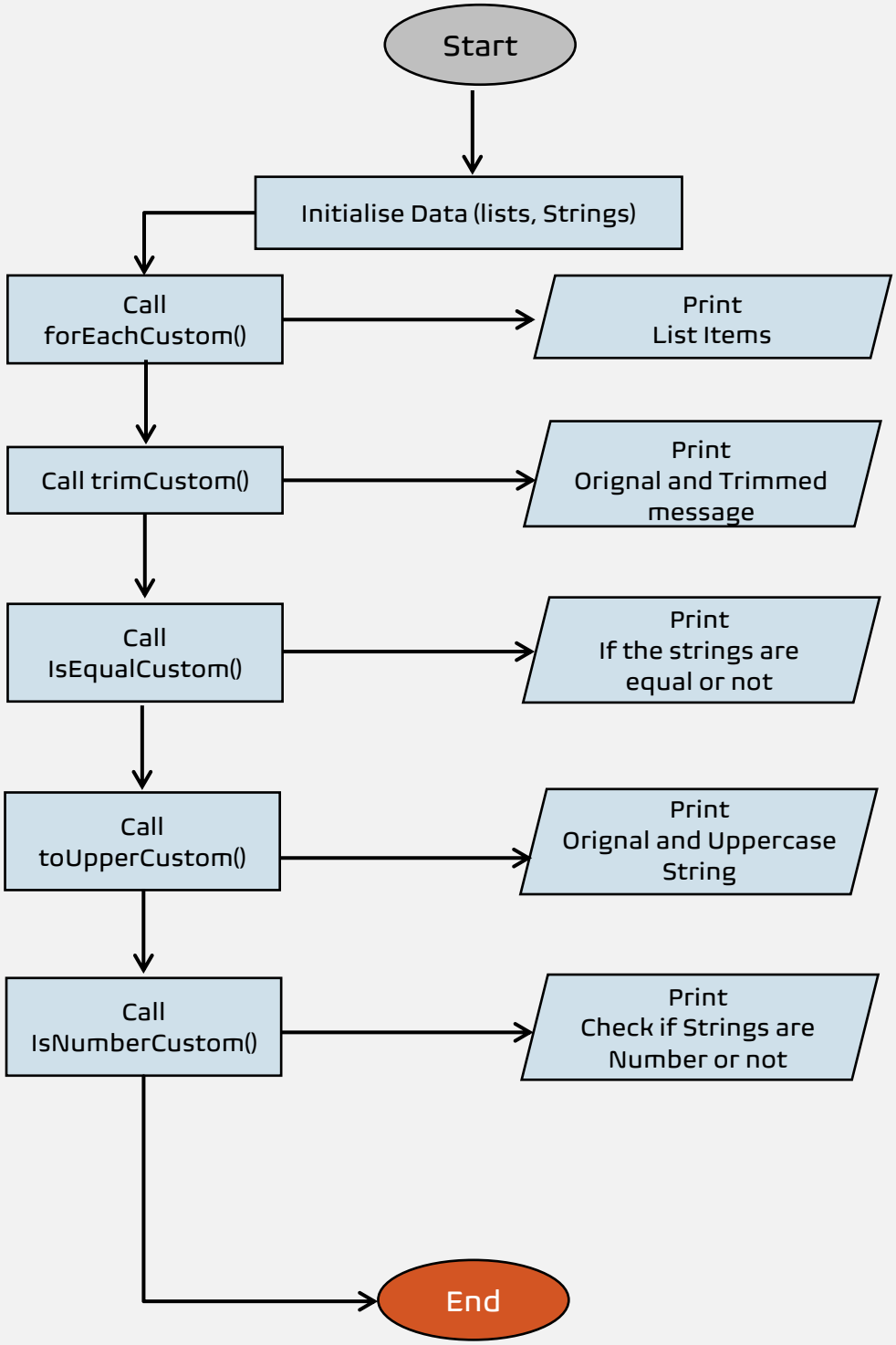
IV. Define the function toUpperExample(): Accept a string and convert it to uppercase using toUpperCase(), then return the result.

V. Define the function isNumberExample(): Accept a string, try to parse it as a double using Double.parseDouble(), and return true if parsing succeeds, otherwise false.

VI. In main(): Create a list of strings and test the forEachExample() function by printing the elements of the list.

VII. In main(): Test the other functions (trimExample(), isEqualExample(), toUpperExample(), and isNumberExample()) by providing sample input and displaying the results.

FlowChart:


Start
Initialise Data (lists, Strings)
Call forEachCustom()
Print List Items
Call trimCustom()
Print Orignal and Trimmed message
Call IsEqualCustom()
Print If the strings are equal or not
Call toUpperCustom()
Print Orignal and Uppercase String
Call IsNumberCustom()
Print Check if Strings are Number or not
End

```java
import java.util.Arrays;
import java.util.List;
public class StringOp {

    public static void forEachCustom(List<String> list) {
        for (String i : list) {
            System.out.println(i);
        }
    }

    public static String trimCustom(String str) {
        return str.trim();
    }

    public static boolean isEqualCustom(String str1, String str2) {
        return str1.equals(str2);
    }

    public static String toUpperCustom(String str) {
        return str.toUpperCase();
    }

    public static String isNumberCustom(String str) {
        try {
            Double.parseDouble(str);
            return "Yes";
        } catch (NumberFormatException e) {
            return "No";
        }
    }

    public static void main(String[] args) {
        // Custom usage of the methods
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
        System.out.println("forEach Custom:");
        forEachCustom(names);

        String message = "  Hello, World!  ";
        System.out.println("\ntrim Test:");
        System.out.println("Original:\"" + message + "\"");
        System.out.println("Trimmed: \"" + trimCustom(message) + "\"");

        String str1 = "Hello";
        String str2 = "Hello";
        System.out.println("\nisEqual Test:");
        System.out.println("Are the strings equal? " + isEqualCustom(str1, str2));

        String lowercase = "hello";
        System.out.println("\ntoUpper Test:");
        System.out.println("Original: " + lowercase);
        System.out.println("Uppercase: " + toUpperCustom(lowercase));

        String str3 = "123.45";
        String str4 = "Hello123";
        System.out.println("\nisNumber Test:");
        System.out.println("Is '" + str3 + "' a number? " + isNumberCustom(str3));
        System.out.println("Is '" + str4 + "' a number? " + isNumberCustom(str4));
    }
}
```

OutPut:

```
forEach Custom:
Alice
Bob
Charlie

trim Test:
Original:"  Hello, World!  "
Trimmed: "Hello, World!"

isEqual Test:
Are the strings equal? true

toUpper Test:
Original: hello
Uppercase: HELLO

isNumber Test:
Is '123.45' a number? Yes
Is 'Hello123' a number? No
```

Variable Declaration Table:

Variable Name	Data Type	Description
names	List <String>	Holds a list of names to perform the custom forEachCustom()
message	String	A string that have Space used in trimCustom()
str1	String	A string used for comparison in isEqualCustom()
str2	String	
lowercase	String	A string that converted in toUpperCustom()
str3	String	Strings that are used to represent Number and non-Number String in isNumberCustom()
str4	String	

8)-> Write a Java program to implement multiple inheritance in case

a. Showcase inheritance

b. Illustrate a try catching.

Algorithm:

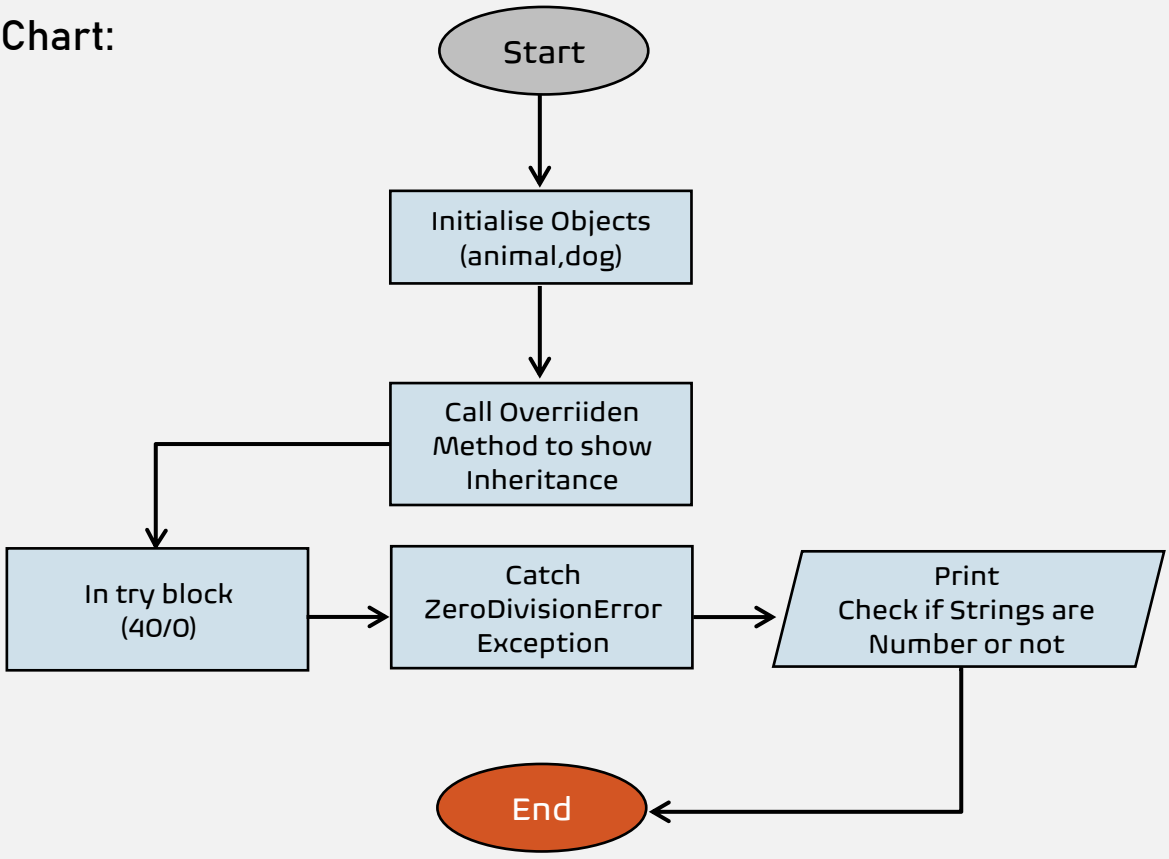
I. Define a parent class Animal with its particular attribute.

II. Create a child class (Dog) that inherit that attribute

III. Use a Try to divide 40 by 0

IV. Catch The ZeroDivisionError exeption.

FlowChart:



```java
public class Inheritance {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Dog dog = new Dog();
        animal.makeSound(); // Output: Some generic animal sound
        dog.makeSound();    // Output: Woof
        System.out.print("\nTrying Try-Catch Block \n\nIn Try Block: ");
        System.out.println("Dividing 40 by 0\n");
        try {
            int result = 40 / 0;
            System.out.println("Result: " + result);
        } catch (Exception e) {
            System.out.print("Caught an exception: ");
            System.out.println(e.getMessage() + "\n\nError: Division by zero is not
allowed.\n");
        }
    }
}
class Animal {
    void makeSound() {
        System.out.print("\nSome generic animal sound : ");
    }
}
class Dog extends  Animal {
    @Override
    void makeSound() {
        System.out.println("Woof.");
    }
}
```

OutPut:

```
Some generic animal sound : Woof.

Trying Try-Catch Block

In Try Block: Dividing 40 by 0

Caught an exception: / by zero

Error: Division by zero is not allowed.
```

Variable Declaration Table:

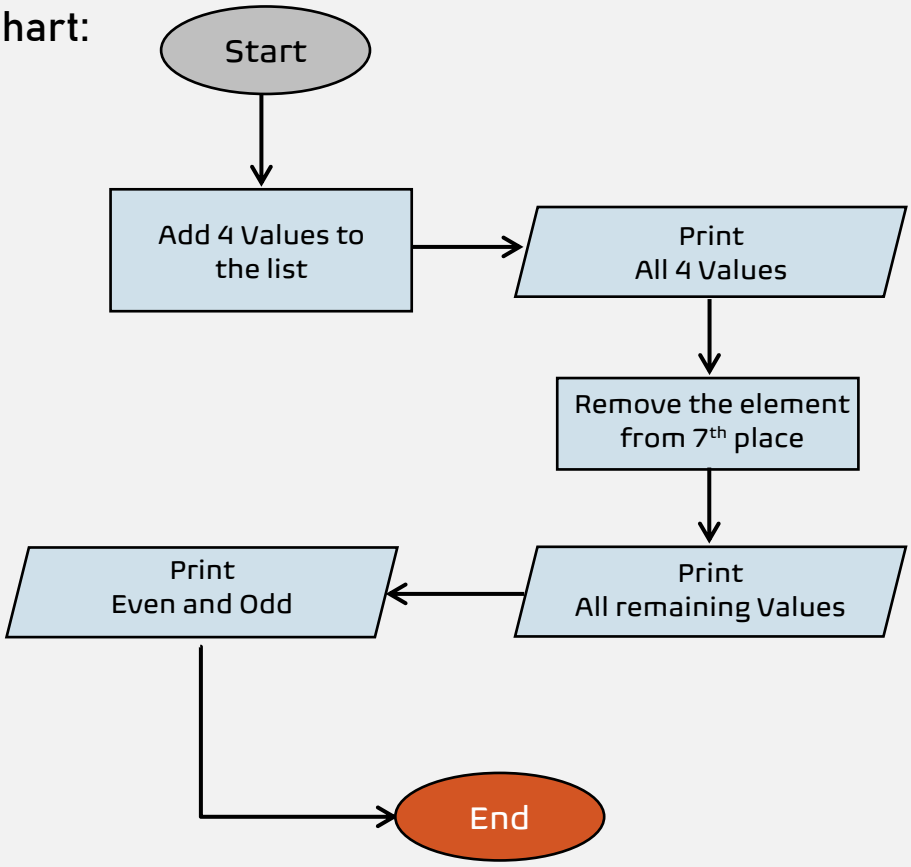
Variable Name	Data Type	Description
animal	Animal	Object of Animal Class the Parent class
dog	Dog	Object of Dog Class the Child class
result	int	stores the result of 40/0

9)-> Write a program in java to create circular Linkedlist of 4 values remove 7th value and print rest of values then print even and odd values respectively.

Algorithm:

I. Initialise CLL: Create Node class, head = null
II. Add Node : If Head node is null, set newNode.next=newNode; else traverse to the last node and assign next value as head
III. Remove Node: if position = 1,update head and adjust last node; else traverse to -1 and skip target node
IV. Print list: Taverse from head to print each value
V. Print Even/Odd Values
VI. Edge Handling: skip operations on empty list or invalid positions
VII. Main Execution: Test all Functions

FlowChart:



```
public class CircularLinkedList {       // Node class representing each element in the
list
    static class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }
    private Node head = null;   // Head of the list
        public void addNode(int data) {  // Method to insert nodes into the Circular
Linked List
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            newNode.next = head; // Circular connection
        } else {
            Node temp = head;
            while (temp.next != head) {
                temp = temp.next;
            }
            temp.next = newNode;
            newNode.next = head; // Circular connection
        }
    }
    public void removeNode(int position) {       // Method to remove the node at the
nth position
        if (head == null) return;
        Node temp = head;
        if (position == 1) {  // Removing the head

 while (temp.next != head) {
                temp = temp.next;
            }
            temp.next = head.next;
            head = head.next;
            return;
        }
        for (int i = 1; temp != null && i < position - 1; i++) {   // Traverse to the
node just before      the node to be deleted
         temp = temp.next;
        }
        if (temp == null || temp.next == null) return;   // If the position is out of
bound
        Node next = temp.next.next;            // Remove the node
        temp.next = next;
    }
    public void printList() {       // Method to print the entire Circular Linked List
        if (head == null) return;
        Node temp = head;
        do {
            System.out.print(temp.data + " ");
            temp = temp.next;
        } while (temp != head);
     System.out.println();
    }
    public void printEvenOdd() {       // Method to print even and odd values
separately
        if (head == null) return;
        Node temp = head;
```

```
        System.out.print("Even values: ");
        do {
            if (temp.data % 2 == 0) {
                System.out.print(temp.data + " ");
            }
            temp = temp.next;
        } while (temp != head);
        System.out.println();

        temp = head;
        System.out.print("Odd values: ");
        do {
            if (temp.data % 2 != 0) {
                System.out.print(temp.data + " ");
            }
            temp = temp.next;
        } while (temp != head);
        System.out.println();
    }
    public static void main(String[] args) {
        CircularLinkedList cll = new CircularLinkedList();
        cll.addNode(1);     // Adding 4 values to the Circular Linked List
        cll.addNode(2);
        cll.addNode(3);
        cll.addNode(4);
        // Print the circular linked list before removing any node
        System.out.println("Circular Linked List before removal:");
        cll.printList();
        cll.removeNode(7);    // Remove the 7th node (though we only have 4, so this is
just a test case)
        // Print the circular linked list after removing the 7th node
        System.out.println("Circular Linked List after removing 7th node:");
        cll.printList();
        // Print even and odd values
        cll.printEvenOdd();
    }

}
```

OutPut:

```
Circular Linked List before removal:
1 2 3 4
Circular Linked List after removing 7th node:
1 2 4
Even values: 2 4
Odd values: 1
```

Variable Declaration Table:

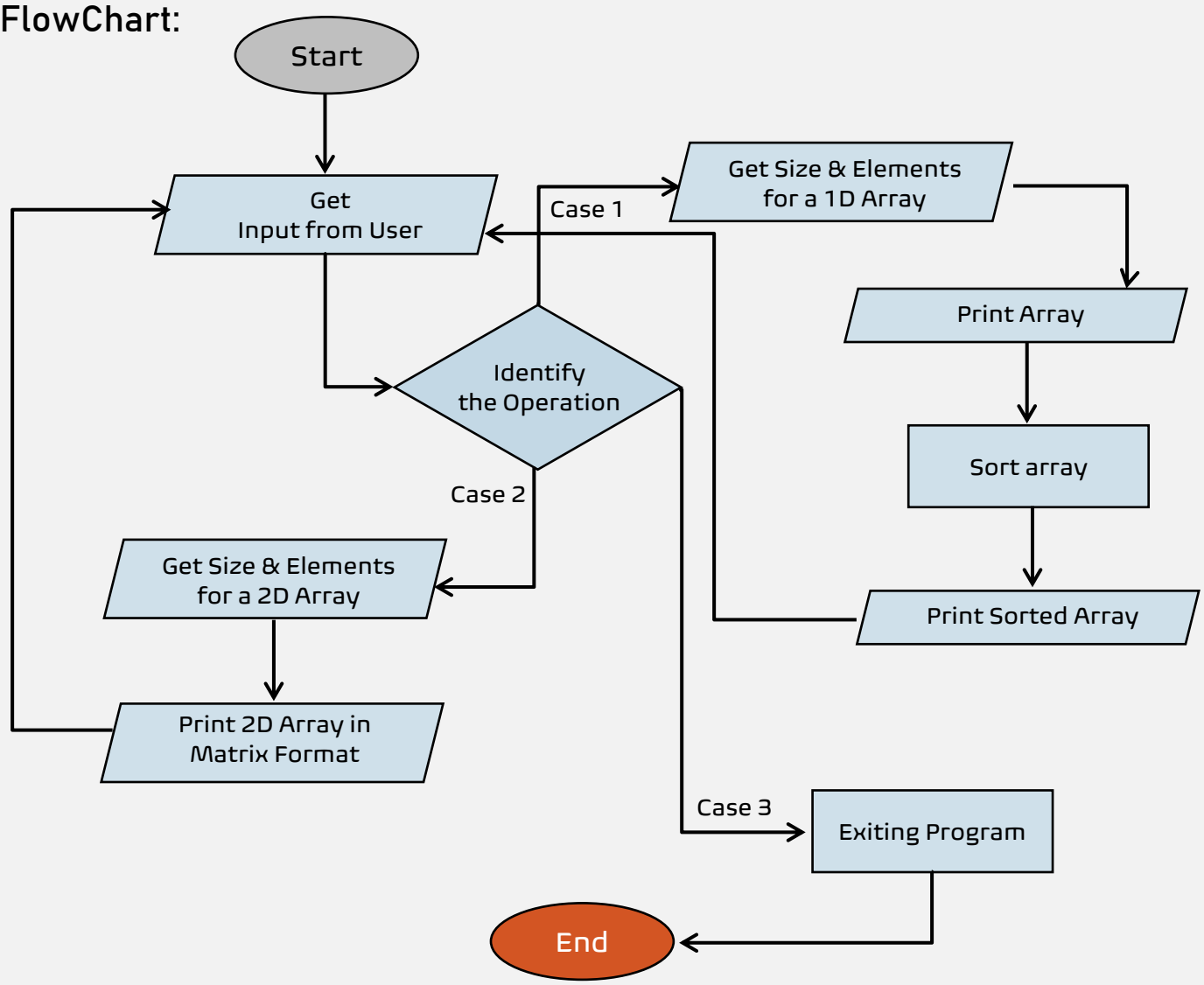
Variable Name	**Data Type**	**Description**
head	Node	Head of linked list
newNode	N ode	Data Stored in each node
next	Node	Reference to next node
temp	Node	Temporary Node reference
position	int	To control the loop
data	int	To control sorting loop
i	int	Temporary node to swapping

10)-> Write a program in java get the value of 1D and 2D array from user using switch case including declaration and initialization and access of array element printing in ascending order.

Algorithm:

I. Display Menu: Show option for 1D Array, 2D Array and Exit
II. Input Choice: Read user input for selectig a option(1, 2, Or 3)
III. Case 1 = 1D Array:
   - Input Size and Element for 1D Array
   - Print the array and its sorted Array
IV. Case 2 = 2D Array:
   - Input rows & columns for 2D Array, then put all the elements
   - Print 2D Array in Matrix Format
V. Case 3 = Exit: Exit program when user chooses to exit
VI. Repeat Menu: Loop to Emnu until user chooses to Exit(Choice 3)

FlowChart:



```java
import java.util.Scanner;
import java.util.Arrays;

public class ArrayDemo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {     // Display menu to user
            System.out.println("\nSelect an option:");
            System.out.println("1. Work with 1D Array");
            System.out.println("2. Work with 2D Array");
            System.out.println("3. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    handleOneDArray(sc);    // Working with 1D Array
                    break;
                case 2:
                   handleTwoDArray(sc);  // Working with 2D Array
     break;

                case 3:
                    System.out.println("Exiting program...");
                    break;
                default:
                    System.out.println("Invalid choice, please try again.");
            }
        } while (choice != 3);
        sc.close();
    }

    public static void handleOneDArray(Scanner sc) {       // Method to handle 1D array
operations
        System.out.print("Enter the size of the 1D array: ");
        int size = sc.nextInt();

       int[] arr = new int[size];          // Declare and initialize the 1D array

        // Initialize the array with user input
        System.out.println("Enter " + size + " elements for the 1D array:");
        for (int i = 0; i < size; i++) {
            System.out.print("Element " + (i + 1) + ": ");
            arr[i] = sc.nextInt();
        }
        System.out.println("You entered the 1D array elements:");  // Access and print
elements of the 1D array
        for (int i = 0; i < size; i++) {
            System.out.print(arr[i] + " ");
        }

        Arrays.sort(arr);  // Sort the array in ascending order
        System.out.println("\nSorted 1D array in ascending order:");  // Print the
sorted 1D array
        for (int i = 0; i < size; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println(); // New line after printing
```

```java
    }

    public static void handleTwoDArray(Scanner sc) { // Method to handle 2D array
operations
        System.out.print("Enter the number of rows for the 2D array: ");
        int rows = sc.nextInt();
        System.out.print("Enter the number of columns for the 2D array: ");
        int cols = sc.nextInt();

        int[][] arr = new int[rows][cols]; // Declare and initialize the 2D array

        System.out.println("Enter elements for the 2D array:");  // Initialize the
array with user input
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print("Element [" + i + "][" + j + "]: ");
                arr[i][j] = sc.nextInt();
            }
        }

        // Access and print the elements of the 2D array
        System.out.println("You entered the 2D array elements:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println(); // Move to next line after each row
        }
    }
}
```

OutPut:

```
Enter your choice: 1
Enter the size of the 1D array: 4
Enter 4 elements for the 1D array:
Element 1: 32
Element 2: 12
Element 3: 25
Element 4: 45
You entered the 1D array elements:
32 12 25 45
Sorted 1D array in ascending order:
12 25 32 45

Select an option:
1. Work with 1D Array
2. Work with 2D Array
3. Exit
Enter your choice: 
```

```
Select an option:
1. Work with 1D Array
2. Work with 2D Array
3. Exit
Enter your choice: 2
Enter the number of rows for the 2D array: 2
Enter the number of columns for the 2D array: 3
Enter elements for the 2D array:
Element [0][0]: 35
Element [0][1]: 62
Element [0][2]: 12
Element [1][0]: 45
Element [1][1]: 32
Element [1][2]: 12
You entered the 2D array elements:
35 62 12
45 32 12

Select an option:
1. Work with 1D Array
2. Work with 2D Array
3. Exit
Enter your choice: 3
Exiting program...
```

Variable Declaration Table:

Variable Name	Data Type	Description
sc	Scanner	A Scanner Object used for reading user Input
choice	int	Stores User’s Menu Choice
size	int	Stores User’s input for Size of 1D Array
arr	int[]	A 1D integer Array used to store elements entered by user
rows	int	Stores the number of Rows for 2D Array
cols	int	Stores the number of Columns for 2D Array
arr	int[][]	A 2D Array used to store elements entered by user in a Matrix
i	int	Loop counter for iterating through Rows or elements of the Array
j	int	Loop counter for iterating through Columns of the Array